Les Réseaux de Neurones Artificiels

Artificial Neural networks (ANN)

Contenu du cours

- Introduction
  - Historique, neurone biologique, neurone artificiel
  - Rappel mathématique (minimisation des fonctions)

- Chapitre 2: Modélisation par les réseaux de neurones artificiels
  - Rappel mathématique
  - Modèle du neurone artificiel
  - Apprentissage supervisé
  - Réseau de neurone à une couche
  - Réseau de neurone multi couches (MLP)
  - Algorithme de rétro-propagation

- Chapitre 3: Commande neuronale à minimum de variance
  - Critère à minimiser
  - Calcul du prédicteur à un pas
  - Calcul de la commande
  - Exemple de simulation

Introduction

Réseaux de neurones Artificiels (RNA)

- ✓ 1943: Travaux de Mac-Culloch et Pitts. Proposition du premier modèle inspiré du fonctionnement du neurone biologique.
- ✓ 1949: Lois d'adaptation créées par les systèmes neuronaux (travaux de Hebb).
- ✓ 1958: Définition du perceptron par Rosenblatt.
- ✓ 1976: Modélisation des systèmes non linéaires par les RNA (Grossberg)
- ✓ Réseaux de: Hopfield 1982, Jordan 1986, Elman 1990

Applications: Modélisation, commande, classification, filtrage, prédiction, …

Article à lire: https://spectrum.ieee.org/history-of-ai

Neurone biologique

Dendrite:

collecte des informations

Corps cellulaire:

Traitement des informations

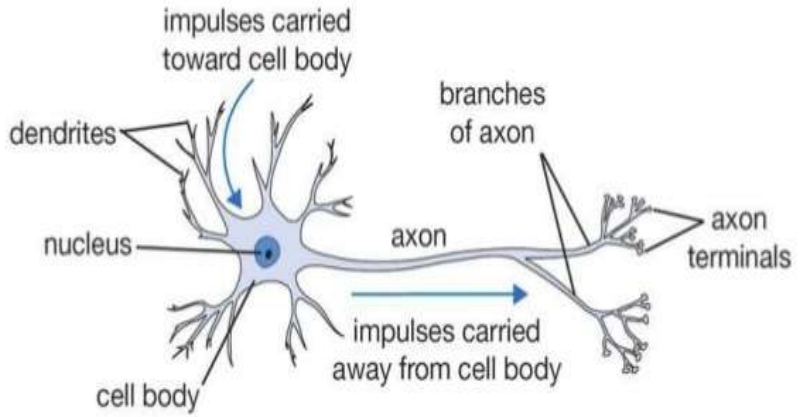
Axone:

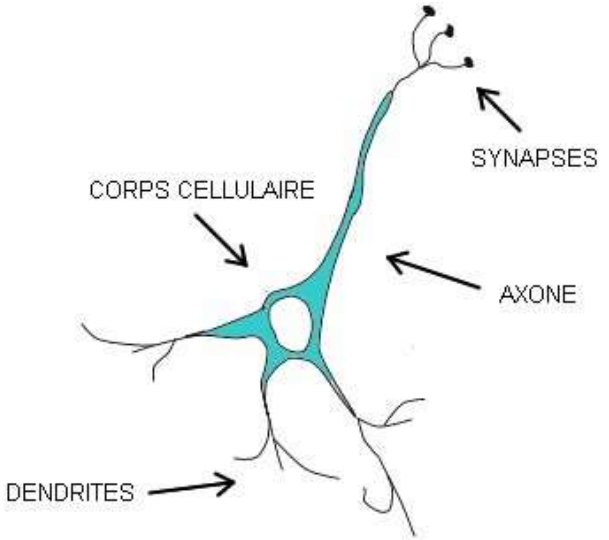
transfert du flux nerveux

Synapses:

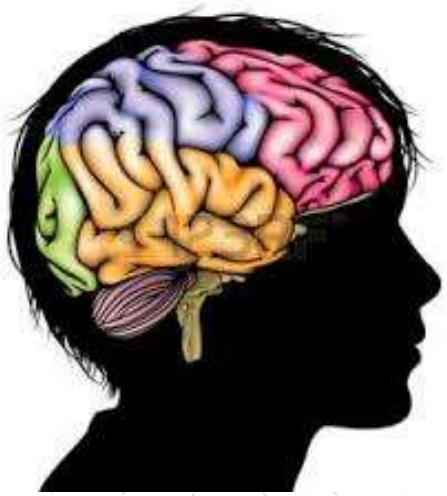
connections avec d'autres neurones.

Neurone biologique





- Le nombre total de neurones du cerveau humain est estimé de 86 à 100 milliards (10^{11}). Il existe plus de 200 types de neurones.







- Apprentissage des neurones biologiques: Répétition
  - Renforcement des liaisons entre certains neurones
  - Affaiblissement des autres liaisons
  - Changement des tailles des synapses

Cerveau humain: très puissant, complexe

Réseau de neurone artificiel: un seul type de neurones, limité, en cours de développement, ???

Neurone artificiel

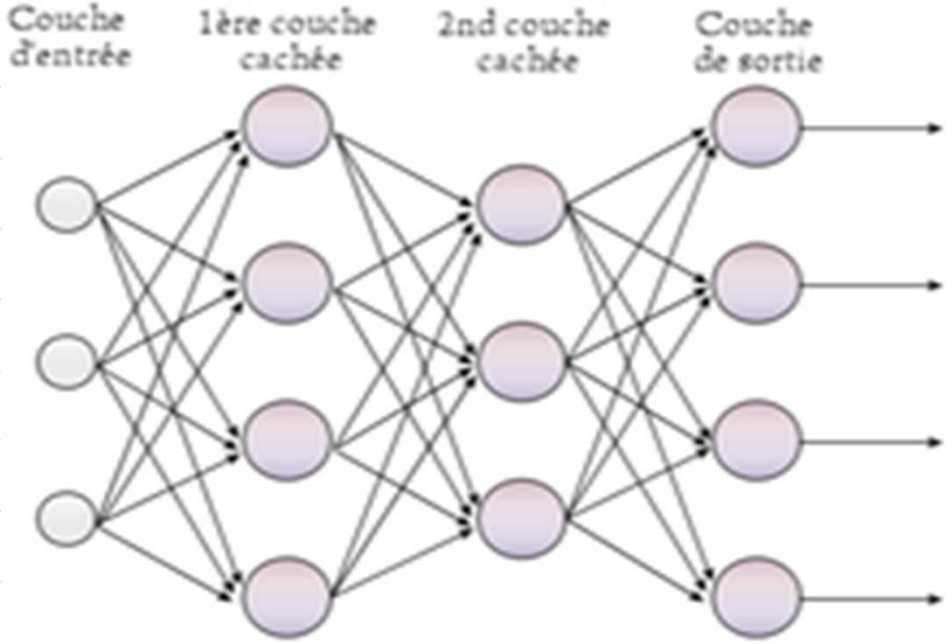
- Formule mathématique = approximation simple du neurone biologique

- Un réseau de neurones artificiels
  - formé du même type de neurones
  - contient plusieurs couches:
    - une couche d'entrée,
    - une ou plusieurs couches cachées,
    - une couche de sortie



Neurones biologiques

- Apprentissage
  - un algorithme mathématique
  - calcul des paramètres du réseau
  - de neurones
- Objectif de l'apprentissage
  - reproduire une tâche (un comportement) bien déterminée.



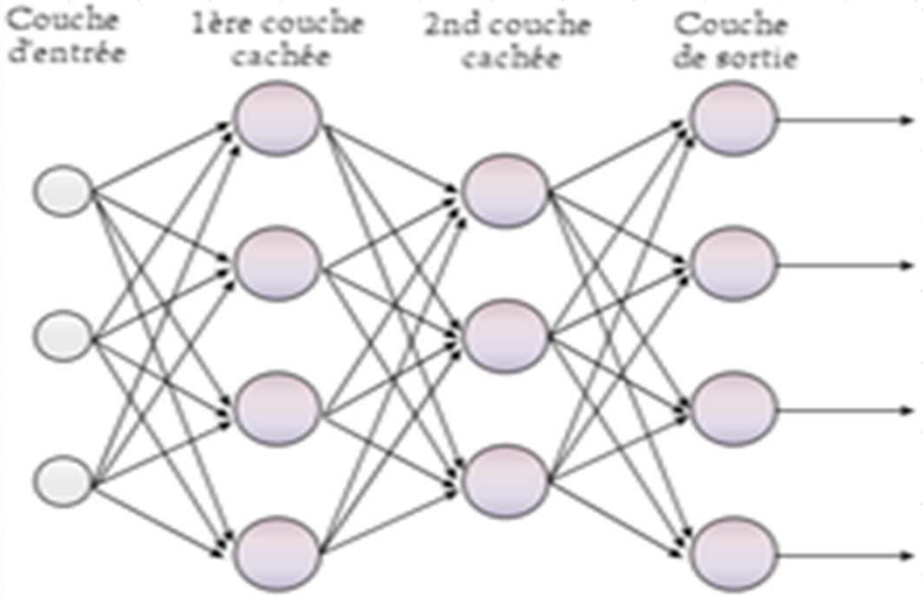
Neurones artificiels

Structure du réseau de neurones artificiels

- Couche d'entrée : contient les entrées du réseau
- Une ou plusieurs couches cachées:
  - Chaque couche contient un certain nombre de neurones
  - Le nombre de couche cachée dépond de la complexité du problème à traiter
  - Généralement, une ou deux couches cachées sont suffisantes
- Une couche de sorties: contient les sorties du réseau

Objectifs du cours

- Comment on utilise les neurones artificiels pour :
  - ✓ La Modélisation et la Commande des systèmes physiques ?



Rappel mathématique

Problème: Calcul de la valeur minimale d'une fonction non linéaire $f(x)$ à une seule variable:

$$\min_x f(x)$$

Méthode de gradient:

- Modification de la variable inconnue d'une quantité proportionnelle au gradient de la fonction et de signe opposé à celui-ci.
- Terme de correction (d'actualisation): $\Delta x = -\mu \frac{\partial f(x)}{\partial x}$

$$0 < \mu \leq 1$$

- Valeur actuelle:

$$x(i) = x(i-1) + \Delta x \qquad i = 1, 2, \ldots, n$$

Valeur optimale: $x_{\min} = x_{optimale} = x(n)$

Fonction minimale:

$$f_{\min} = f(x_{optimale})$$

Algorithme

- Introduction de: μ , x , n et $i=1$
- Boucle de calcul

For i=2:n

calcul de : $\frac{\partial f(x)}{\partial x}$

terme d'actualisation: $\Delta x = -\mu \frac{\partial f(x)}{\partial x}$

actualisation de x: $x = x + \Delta x$

End

Tests d'arrêt: n_{max}

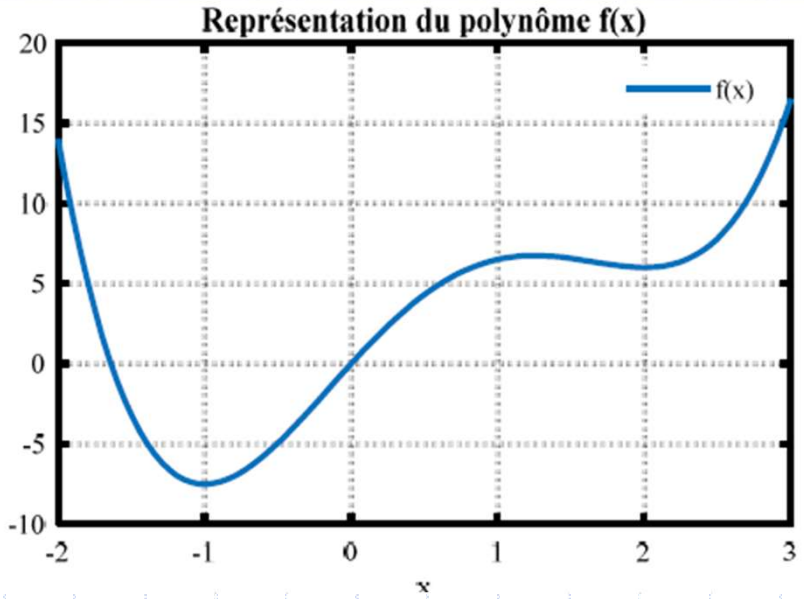
ou la dérivée de $f(x)$ converge vers zéro

Valeur optimale: $x_{\min} = x_{optimale} = x(n)$

Fonction minimale: $f_{\min} = f(x_{optimale})$

Exemple

$$f(x) = x^4 - 3x^3 - 1{,}5x^2 + 10x$$



Fonction non convexe:

2 minimums sur l'intervalle: $x \in [-2, 3]$

$$x_{1\min} = -1, \quad f(x_{1\min}) = -7{,}5$$

$$x_{2\min} = 2, \quad f(x_{2\min}) = 6$$

Méthode de gradient:

$$\Delta x = -\mu \frac{\partial f(x)}{\partial x} = -\mu(4x^3 - 9x^2 - 3x + 10)$$

$$x(i) = x(i-1) + \Delta x \qquad i = 1, 2, \ldots, n$$

x_1	-1,5	0	1,5	2,5
$x_{optimale}$	-1	-1	1,99	1,99
$f(x_{opt})$	-7,5	-7,5	6	6

La solution finale dépend de la valeur initiale (initialisation).

Problème: Calcul de la valeur minimale d'une fonction non linéaire à *m* variables:

$$f(x_1, x_2, \ldots, x_m).$$

Méthode de gradient:

- Terme de correction (d'actualisation):

$$\Delta x_j = -\mu \frac{\partial f(x_j)}{\partial x_j} \qquad j = 1, \ldots, m \qquad 0 < \mu \leq 1$$

- Valeur actuelle:

$$x_j = x_j + \Delta x_j \qquad j = 1, \ldots, m \qquad i = 1, 2, \ldots, n$$

Valeur optimale:

$$x_{j,\min} = x_{j,optimale} = x_j(n)$$

Fonction minimale:

$$f_{\min} = f(x_{optimale})$$

Problème: Calcul de la valeur minimale d'une fonction quadratique à *m* variables:

$$J = \frac{1}{2}\left(h(y) - f(g(x_1, x_2, \ldots, x_m))\right)^2$$

Méthode de gradient:

Boucle i (i=1, …, n)

- Terme de correction: $\Delta x_j = -\mu \frac{\partial J}{\partial x_j} \quad j = 1, \ldots, m \qquad 0 < \mu \leq 1$
- Valeur actuelle: $x_j = x_j + \Delta x_j \qquad j = 1, \ldots, m$

Fin boucle i

Valeur optimale: $x_{j,\min} = x_{j,optimale} = x_j(n)$

Problème: Calcul de la valeur minimale d'une fonction quadratique à *m* variables:

$$J = \frac{1}{2}\left(h(y) - f(g(x_1, x_2, \ldots, x_m))\right)^2$$

Méthode de gradient:

$$\Delta x_j = -\mu \frac{\partial J}{\partial x_j} \quad j = 1, \ldots, m \qquad x_j = x_j + \Delta x_j \quad j = 1, \ldots, m$$

$$\frac{\partial J}{\partial x_j} = \frac{\partial J(x_j)}{\partial f} \frac{\partial f}{\partial g} \frac{\partial g}{\partial x_j}$$

$$\frac{\partial J}{\partial f(.)} = \tfrac{1}{2}\, 2\, (h(y) - f(g(x_1, \ldots, x_m))(-1) \qquad \frac{\partial f(.)}{\partial g(.)} \frac{\partial g(.)}{\partial x_j} = f'\left(g(x_1, \ldots, x_m)\right) g'(x_1, \ldots, x_m)$$

$$\frac{\partial J}{\partial x_j} = -(h(y) - f(g(x_1, \ldots, x_m)) f'\left(g(x_1, \ldots, x_m)\right) \frac{\partial g(x_1, \ldots, x_m)}{\partial x_j}$$

$$x_{optimale} \Rightarrow \frac{\partial J}{\partial x_j} = 0$$

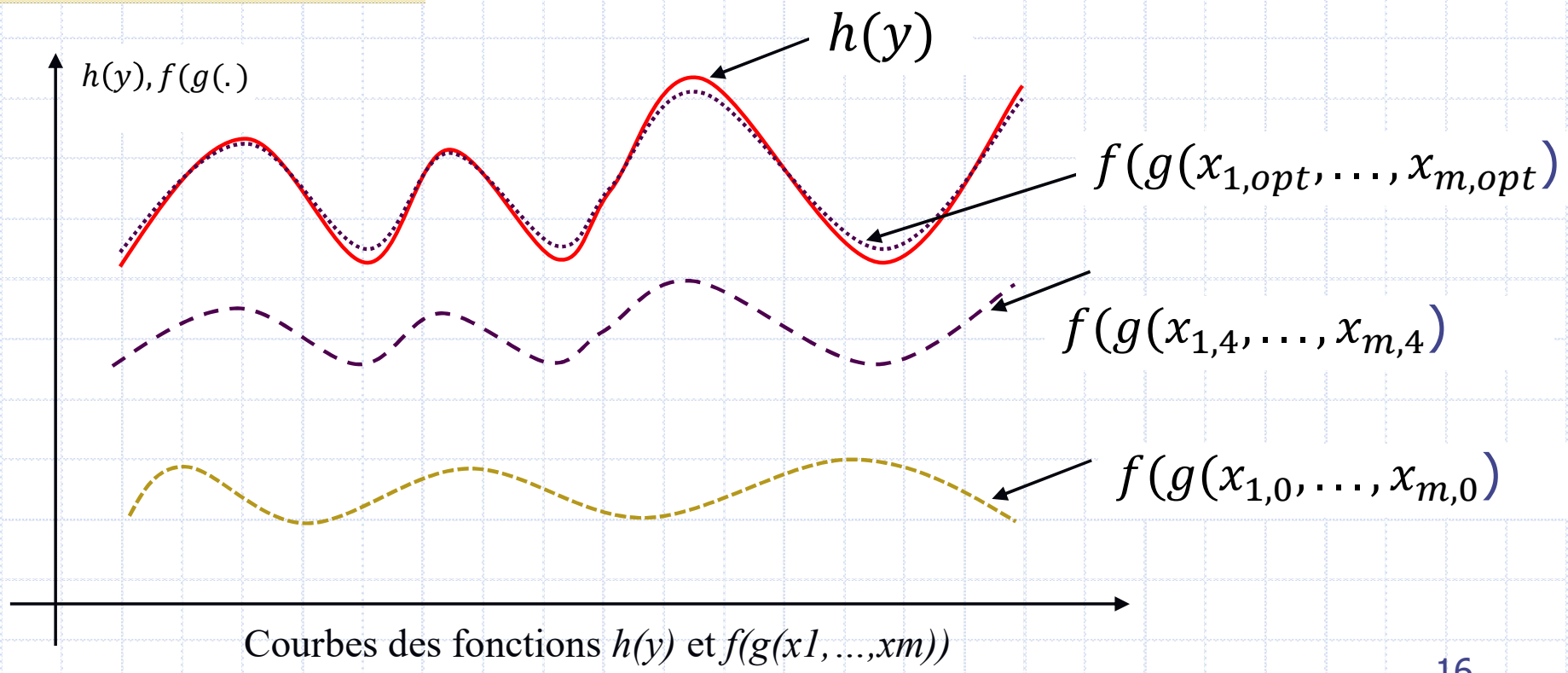
$$f(g(x_{1,opt}, \ldots, x_{m,opt}) = h(y)$$

Problème: Calcul de la valeur minimale d'une fonction quadratique à *m* variables:

$$J = \frac{1}{2}\left(h(y) - f(g(x_1, x_2, \ldots, x_m))\right)^2$$

$$x_{optimale} \Rightarrow \frac{\partial J}{\partial x_j} = 0$$

$$f(g(x_{1,opt}, \ldots, x_{m,opt}) = h(y)$$



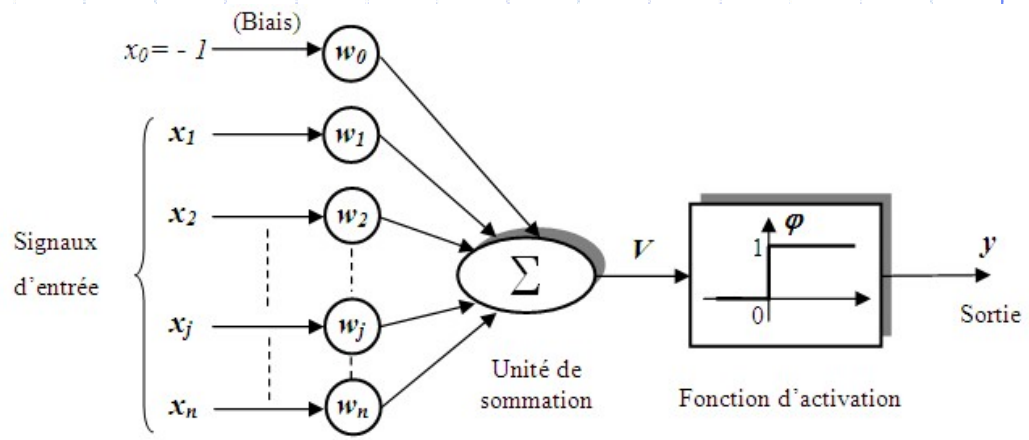
Courbes des fonctions *h(y)* et *f(g(x1,...,xm))*

Chapitre 2

Modélisation par les réseaux de neurones artificiels

Modèle de Mac-Culloch et Pitts

Equations



x_i : i^{ème} entrée.

w_0 : biais ou seuil d'activation du neurone.

W_i : poids des connexions (pondération)

$$v = \sum_{i=0}^{n} w_i x_i = -w_0 + w_1 x_1 + \ldots + w_n x_n$$

Sortie du neurone:

$$y = f(v)$$

Fonction d'activation du neurone:

$$f(v) = \begin{cases} 1 & v \geq 0 \\ 0 & v < 0 \end{cases}$$

Ce modèle peut être utilisé pour réaliser les fonctions logiques (ET, OU, Non, NAND, NOR, ...).

Cas général

- Equations

x_i : $i^{ème}$ entrée.

w_0 : biais ou seuil d'activation du neurone.

$$v = \sum_{i=0}^{n} w_i x_i = -w_0 + w_1 x_1 + \ldots + w_n x_n$$

Sortie du neurone: $y = f(v)$

f : la fonction d'activation du neurone.

f : fonction non linéaire.

Modèle permettant de traiter des données réelles.

Article à lire:

https://spectrum.ieee.org/what-is-deep-learning/particle-17

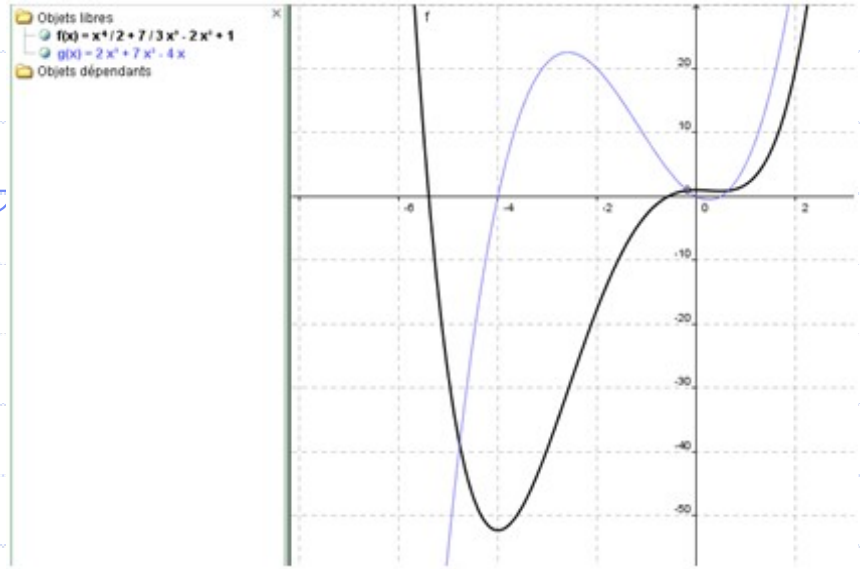
Modèle du neurone artificiel

$$v = \sum_{i=0}^{n} w_i x_i = -w_0 + w_1 x_1 + \ldots + w_n x_n$$

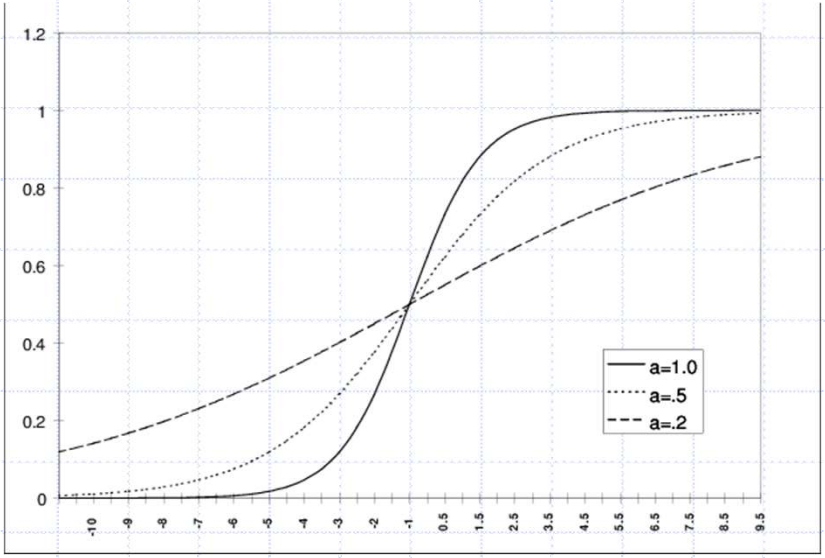
Sortie du neurone: $y = f(v)$

Fonction d'activation du neurone:

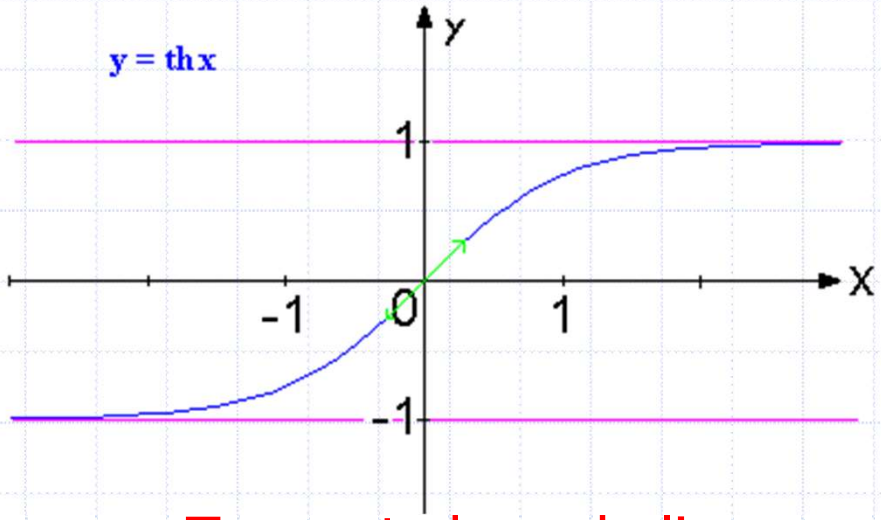
- ✓ Polynômiale (valeurs réelles) $f(v) = \sum_{i=0}^{n} a_i v^i = a_0 + a_1 v + \cdots + a_n v^n$
- ✓ Tangente sigmoïde (valeurs entre 0 et 1) $f(v) = \frac{1}{1 + e^{-2\beta v}}$
- ✓ Tangente hyperbolique (valeurs entre -1 et 1) $f(v) = \frac{1 - e^{-2\beta v}}{1 + e^{-2\beta v}}$
- ✓ Radiale de base (valeurs entre 0 et 1) $f(v) = e^{-\frac{(v-c)^2}{\sigma^2}}$



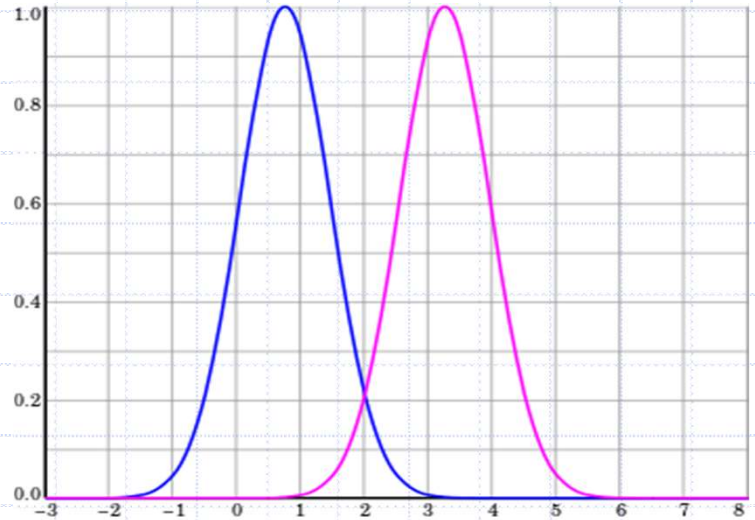
fonction polynômiale



fonction tangente sigmoïde



Tangente hyperbolique



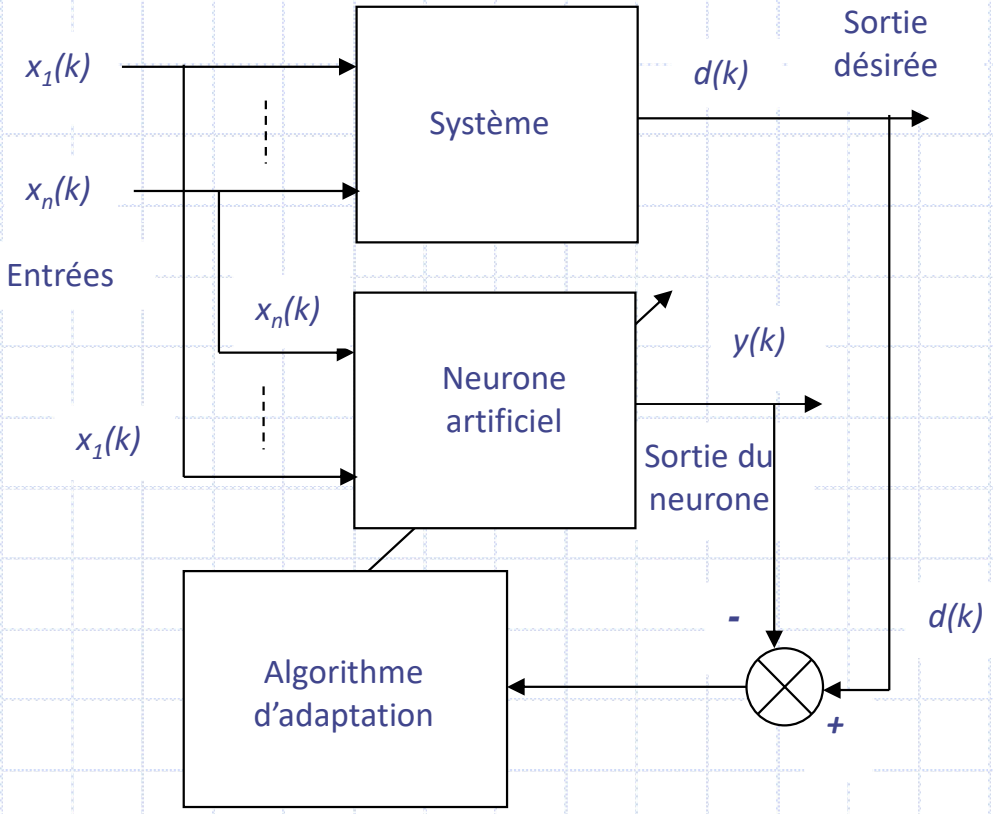
Fonction radiale de base

Neurone artificiel à fonction d'activation non linéaire:

- Modèle non linéaire
- Traitement des données réelles

Apprentissage: détermination des poids de connexions

$$w_i \ (i = 0,1,\ldots,n)$$



Critère à minimiser:

$$J = \frac{1}{2}\left(d(k) - y(k)\right)^2$$

$d(k)$: sortie désirée

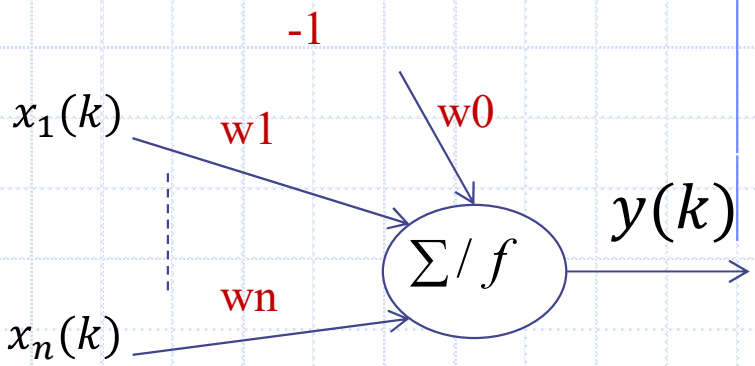
$y(k)$: sortie du neurone

Exemple d'un seul neurone à n entrée et une sortie

$$v(k) = \sum_{i=0}^{n} w_i x_i(k)$$

$$= -w_0 + w_1 x_1(k) + \ldots + w_n x_n(k)$$

$$y(k) = f(v(k))$$

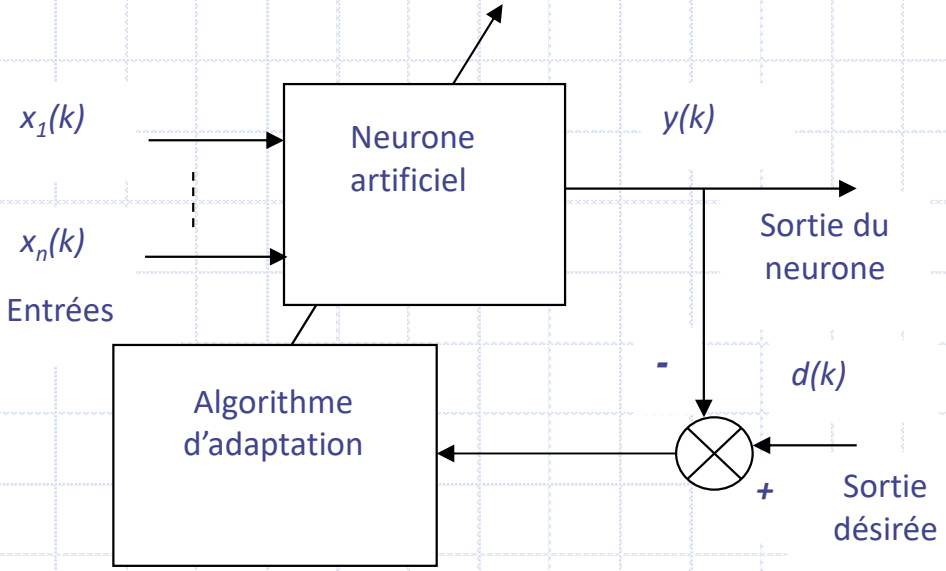


Critère à minimiser:

$$J = \frac{1}{2}\left(d(k) - y(k)\right)^2$$

$d(k)$: sortie désirée

$y(k)$: sortie du neurone



Algorithme de gradient:

- Modification des paramètres (pondération, poids) d'une quantité proportionnelle au gradient du critère J et de signe opposé à celui-ci.
- Terme de correction (d'actualisation): $\Delta w_i = -\mu \frac{\partial J}{\partial w_i}$
- Valeur actuelle: $w_i(k) = w_i(k-1) + \Delta w_i$

Calcul de Δw_i

$$J = \frac{1}{2}\left(d(k) - y(k)\right)^2$$

$$\frac{\partial J}{\partial y(k)} = -(d(k) - y(k))$$

$$\Delta w_i = -\mu \frac{\partial J}{\partial y(k)} \frac{\partial y(k)}{\partial v(k)} \frac{\partial v(k)}{\partial w_i} = \mu (d(k) - y(k)) \frac{\partial y(k)}{\partial v(k)} \frac{\partial v(k)}{\partial w_i}$$

$$y(k) = f(v(k)) \qquad v(k) = w_0 x_0(k) + w_1 x_1(k) + \ldots + w_n x_n(k)$$

$$x_0(k) = -1, \ \forall k$$

$$\Delta w_i = \mu (d(k) - y(k)) f'(v(k)) x_i(k) \qquad i = 0, \ldots, n$$

Algorithme d'apprentissage

L'apprentissage des réseaux de neurones artificiels (détermination des poids de connexions) est basé sur la répétition des mesures.

On utilise deux boucles:

- Boucle de passages $p = 1, \dots, maxp$
- Boucle de mesures $k = 1, \dots, N$

Chaque passage, on traite toutes les mesures.

On calcule, chaque passage, le critère de performance:

$$C(p) = \sum_{k=1}^{N} (d(k) - y(k))^2$$

Ce critère décroit et converge vers une valeur très faible.

Algorithme d'apprentissage

Initialisation:

N: nombre de mesures, maxp: nombre de passages

w_i : valeurs faibles et aléatoire des paramètres du neurone

μ : pas d'adaptation

Préparation des données: $x_1(k)$ … $x_n(k)$ $d(k)$ $k = 1, \ldots, N$

Boucle de passage (p=1: maxp)

Boucle de mesures (k=1:N)

- Calcul de la sortie du neurone suite à la présentation des entrées

$$v(k) = \sum_{i=0}^{n} w_i \, x_i(k) \qquad y(k) = f(v(k))$$

- Modification des pondérations

$$\Delta w_i = \mu(d(k) - y(k)) f'(v(k)) x_i(k)$$

$$w_i(k) = w_i(k-1) + \Delta w_i$$

Fin boucle de mesures

$$C(p) = \sum_{k=1}^{N} (d(k) - y(k))^2$$

Fin boucle de passages

Exemple:

Système: $g(k) = 0{,}4\sin(0{,}2k) + 0{,}6\cos(0{,}4k)$

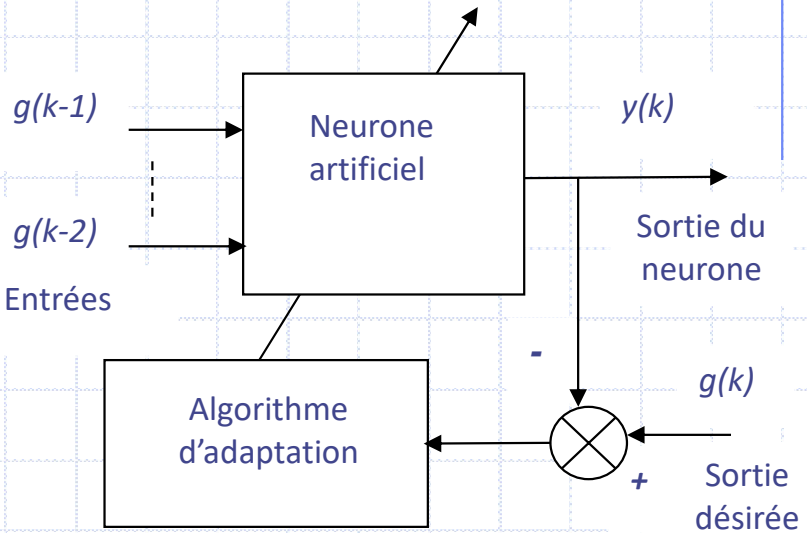
Modèle:

Un seul neurone à deux entrées:

$$x_1(k) = g(k-1) \quad x_2(k) = g(k-2)$$

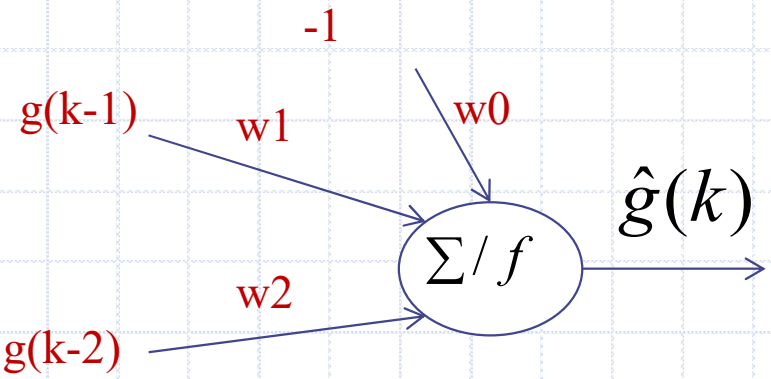
$$v(k) = -w_0 + w_1 g(k-1) + w_2 g(k-2)$$

$$y(k) = \hat{g}(k) = f(v(k))$$



$$J = \frac{1}{2}\left(g(k) - \hat{g}(k)\right)^2$$

$$w_i(k) = w_i(k-1) + \Delta w_i \qquad i = 0,1,2$$



$$\Delta w_0 = \mu(g(k) - \hat{g}(k)) f'(v(k))(-1)$$

$$\Delta w_1 = \mu(g(k) - \hat{g}(k)) f'(v(k)) g(k-1)$$

$$\Delta w_2 = \mu(g(k) - \hat{g}(k)) f'(v(k)) g(k-2)$$

Programme (Octave)

```
clear all
close all
% Package control
pkg load control
% Package optimisation
pkg load optim

N=50;    % nombre de mesures

% Fonction non linéaire g(k)

for k=1:N
      g(k)=0.4*sin(0.2*k)+0.6*cos(0.4*k);
End

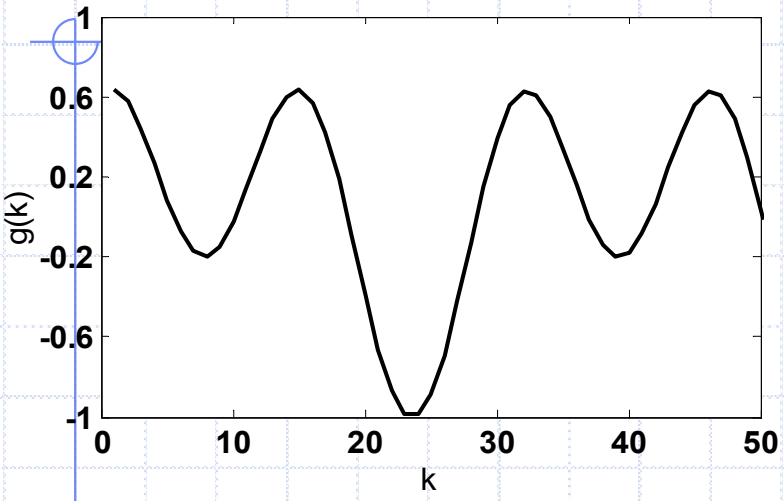
% Initialisation
mu=0.2;          % pas d'adaptation;
% paramètres du neurone artificiel
w0=0.1; w1=-0.1; w2=0.1
pmax=30;        % nombre maximal de passages
% valeurs initiales de la sortie du neurone
y(1)=g(1); y(2)=g(2);
```

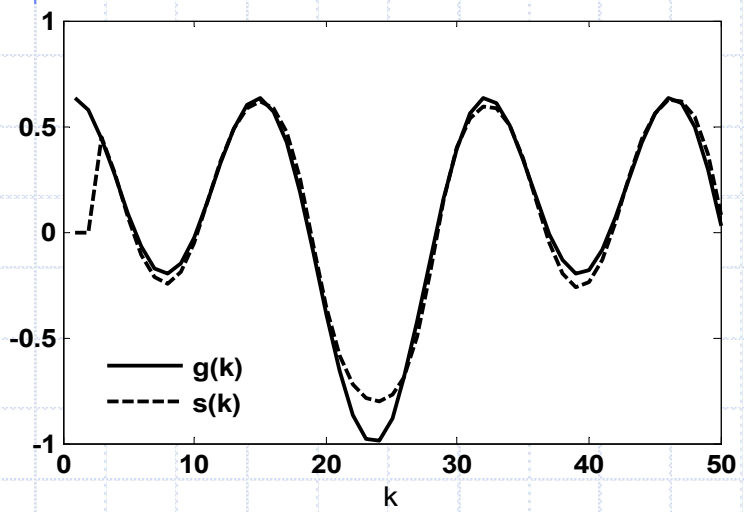
```
% Apprentissage
for p=1:pmax                % Boucle passages
     for k=3:N              % Boucle mesures
          v=-w0+w1*g(k-1)+w2*g(k-2);
          y(k)=tanh(v);        % sortie du neurone
          % Termes de correction
          df = deriv(@tanh, v);  % calcul de tanh'(v)
          dw0=mu*(g(k)-y(k))*df*(-1);
          dw1=mu*(g(k)-y(k))*df*g(k-1);
          dw2=mu*(g(k)-y(k))*df*g(k-2);
          % Adaptation des pondérations
          w0=w0+dw0;
          w1=w1+dw1;
          w2=w2+dw2;
     end                        % Fin boucle de mesures
  % critère : somme des erreurs au carré
  Cr(p)=(g-y)*(g-y)';
end       % Fin boucle de passages

figure(1)
plot(Cr,'*r')    % critère de performance
title('Evolution de la somme des erreurs au carré','fontsize',16)
grid
xlabel('Nombre de passages')
ylabel('Critère Cr')
figure(2)
plot(y);
hold on
plot(g,'g');
grid
title({'Evolutions de g(k) et de y(k)';'0<k<51'},'fontsize',16)
xlabel('itération k')
ylabel('Sortie système et sortie modèle')
```

Résultats

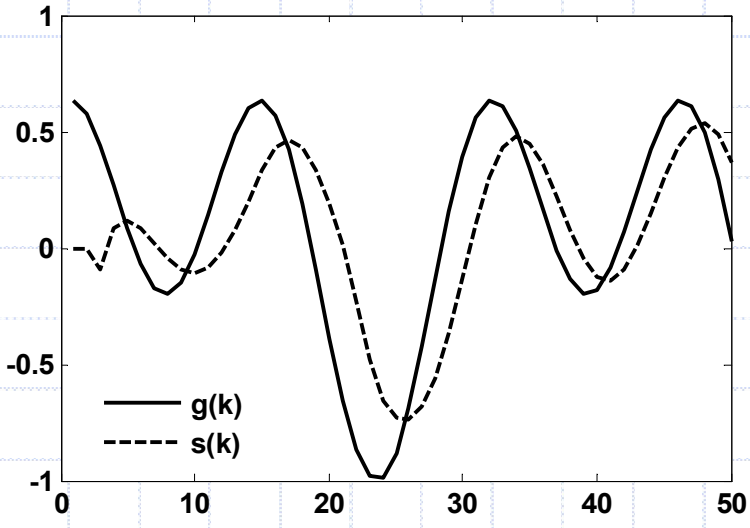
Evolution de g(k) (k=1:50)





Après 30 passages

Evolution de g(k) et la sortie du neurone



Après un seul passage

μ=0.2. Fonction d'activation:

$$f(v(k)) = \frac{1-e^{-2v(k)}}{1+e^{-2v(k)}}$$

Valeurs initiales:

$$w_0 = 0.1,\ w_1 = -0.1,\ w_2 = 0.1$$

Valeurs retrouvées après 30 passages:

$$w_0 = -0.0027,\ w_1 = 2.02,\ w_2 = -1.07$$

Exemple:

Système: $g(k) = 0{,}4\sin(0{,}2k) + 0{,}6\cos(0{,}4k)$

Modèle retrouvé:

$$v(k) = -w_0 + w_1 g(k-1) + w_2 g(k-2) \quad w_0 = -0.0027,\ w_1 = 2.02,\ w_2 = -1.07$$

$$\hat{g}(k) = f(v(k))$$

$$f(v(k)) = \frac{1 - e^{-2v(k)}}{1 + e^{-2v(k)}}$$

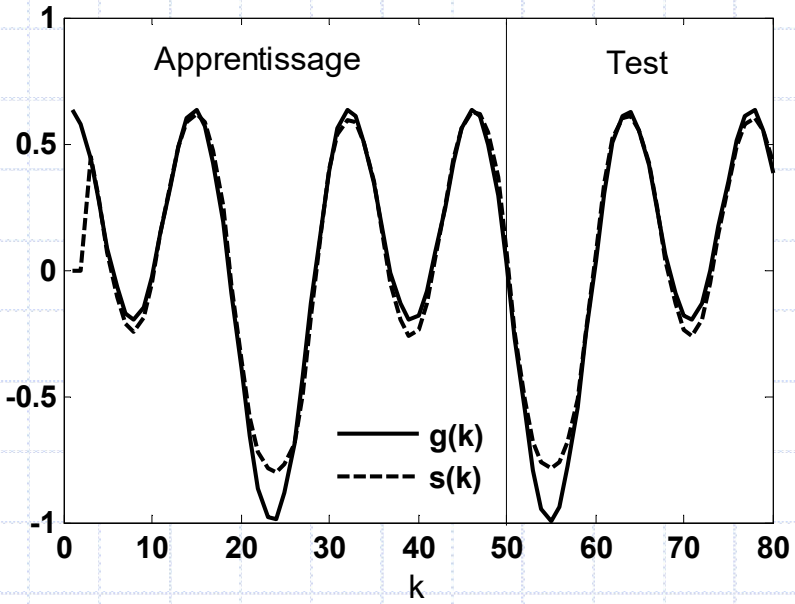
Prédiction:

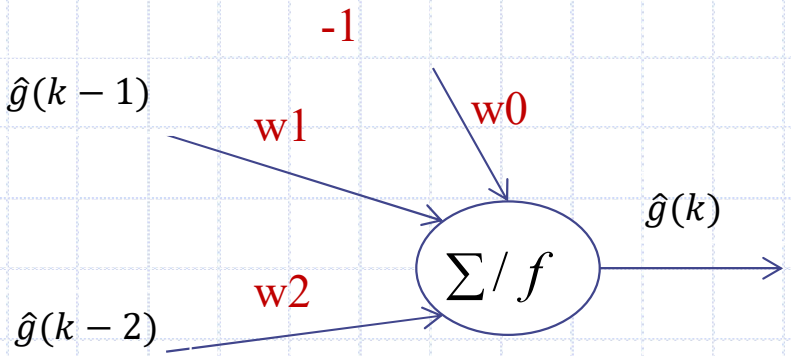
$$\hat{g}(k+1) = f(0.0027 + 2.02 g(k) - 1.07 g(k-1))$$

$$\hat{g}(k+2) = f(0.0027 + 2.02 \hat{g}(k) - 1.07 g(k))$$

$j \geq 3$

$$\hat{g}(k+j) = f(0.0027 + 2.02 \hat{g}(k+j-1) - 1.07 \hat{g}(k+j-2))$$



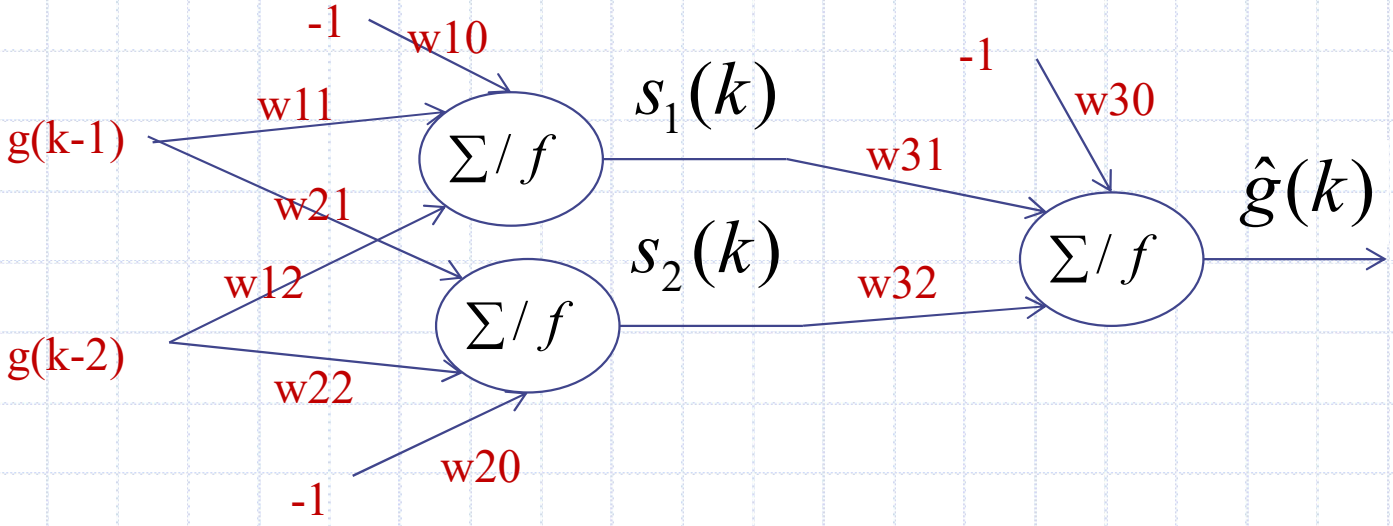


Exemple:

Système: $g(k) = 0{,}4\sin(0{,}2k) + 0{,}6\cos(0{,}4k)$

Modèle: Un réseau de trois neurones

- Entrées: $g(k-1)$ $g(k-2)$
- Couche cachée: 2 neurones
- Couche de sortie: un neurone



Modèle: Un réseau de trois neurones

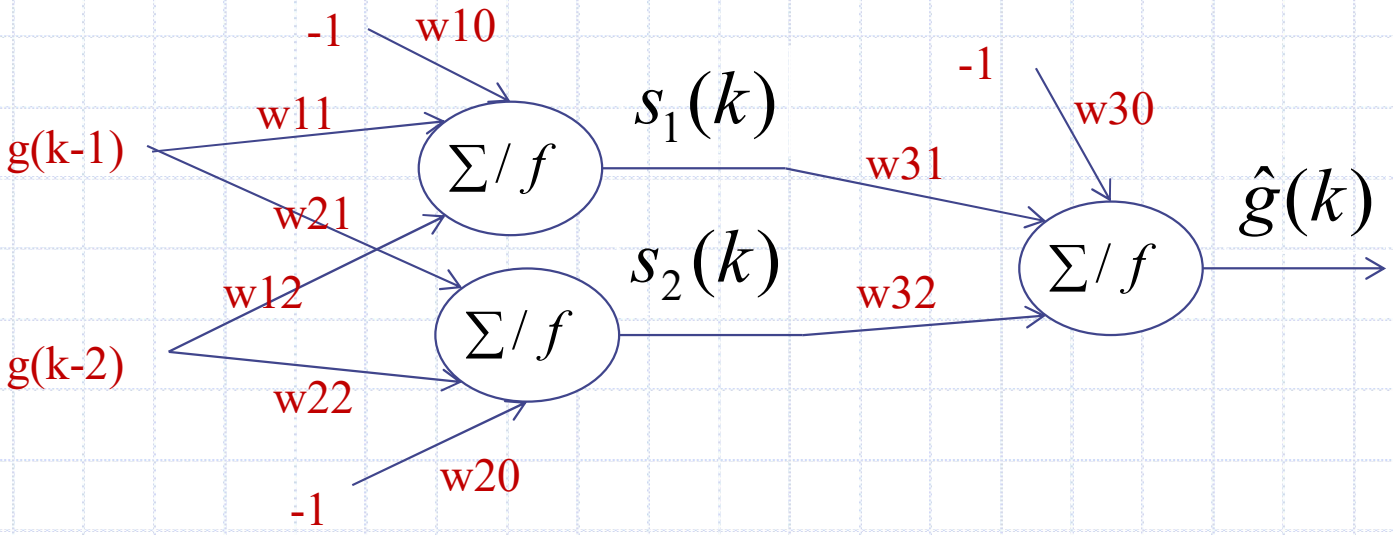
- Entrées: $g(k-1)$ $g(k-2)$
- Sorties de la couche cachée

$$v_1(k) = -w_{10} + w_{11}g(k-1) + w_{12}g(k-2) \qquad s_1(k) = f(v_1(k))$$

$$v_2(k) = -w_{20} + w_{21}g(k-1) + w_{22}g(k-2) \qquad s_2(k) = f(v_2(k))$$

- Sortie de la couche de sortie

$$v(k) = -w_{30} + w_{31}s_1(k) + w_{32}s_2(k) \qquad \hat{g}(k) = f(v(k))$$



Exemple:

Système: $g(k) = 0{,}4\sin(0{,}2k) + 0{,}6\cos(0{,}4k)$

Modèle: Un réseau de trois neurones

Critère à minimiser: $J = \frac{1}{2}\left(g(k) - \hat{g}(k)\right)^2$

Adaptation des paramètres:

$$w(k) = w(k-1) + \Delta w \qquad \Delta w = -\mu \frac{\partial J}{\partial w}$$

Paramètres du neurone de sortie:

$$v(k) = -w_{30} + w_{31}s_1(k) + w_{32}s_2(k) \qquad \hat{g}(k) = f(v(k))$$

$$\Delta w_{3i} = -\mu \frac{\partial J}{\partial w_{3i}} = -\mu \frac{\partial J}{\partial \hat{g}(k)} \frac{\partial \hat{g}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial w_{3i}} \qquad i = 0, 1, 2$$

Adaptation des paramètres du neurone de sortie:

$$\Delta w_{30} = \mu(g(k) - \hat{g}(k)) f'(v(k))(-1)$$

$$\Delta w_{31} = \mu(g(k) - \hat{g}(k)) f'(v(k)) s_1(k)$$

$$\Delta w_{32} = \mu(g(k) - \hat{g}(k)) f'(v(k)) s_2(k)$$

Critère à minimiser: $J = \frac{1}{2}\left(g(k) - \hat{g}(k)\right)^2$

$$w(k) = w(k-1) + \Delta w$$

$$\Delta w = -\mu \frac{\partial J}{\partial w}$$

Paramètres des neurones de la couche cachée:

- Adaptation des paramètres w_{1i}:

$$\Delta w_{1i} = -\mu \frac{\partial J}{\partial w_{1i}} = -\mu \frac{\partial J}{\partial \hat{g}(k)} \frac{\partial \hat{g}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial s_1(k)} \frac{\partial s_1(k)}{\partial v_1(k)} \frac{\partial v_1(k)}{\partial w_{1i}} \qquad i = 0, 1, 2$$

$$\Delta w_{10} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{31} f'(v_1(k))(-1)$$

$$\Delta w_{11} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{31} f'(v_1(k)) g(k-1)$$

$$\Delta w_{12} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{31} f'(v_1(k)) g(k-2)$$

- Adaptation des paramètres w_{2i}:

$$\Delta w_{2i} = -\mu \frac{\partial J}{\partial w_{2i}} = -\mu \frac{\partial J}{\partial \hat{g}(k)} \frac{\partial \hat{g}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial s_2(k)} \frac{\partial s_2(k)}{\partial v_2(k)} \frac{\partial v_2(k)}{\partial w_{2i}} \qquad i = 0, 1, 2$$

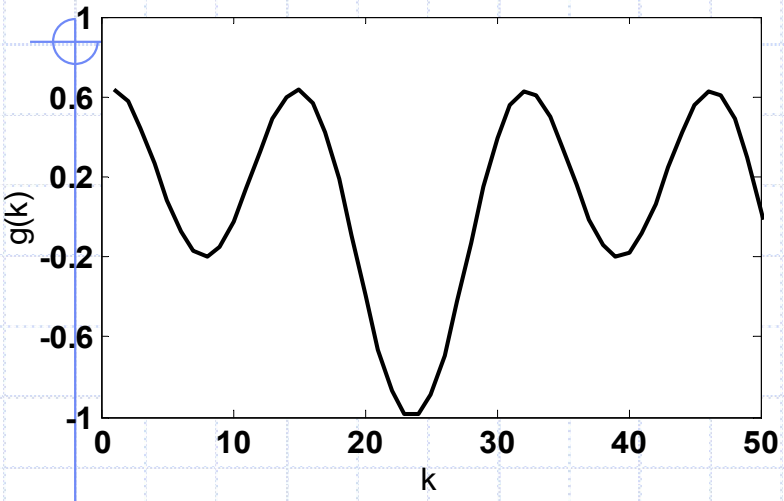
$$\Delta w_{20} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{32} f'(v_2(k))(-1)$$

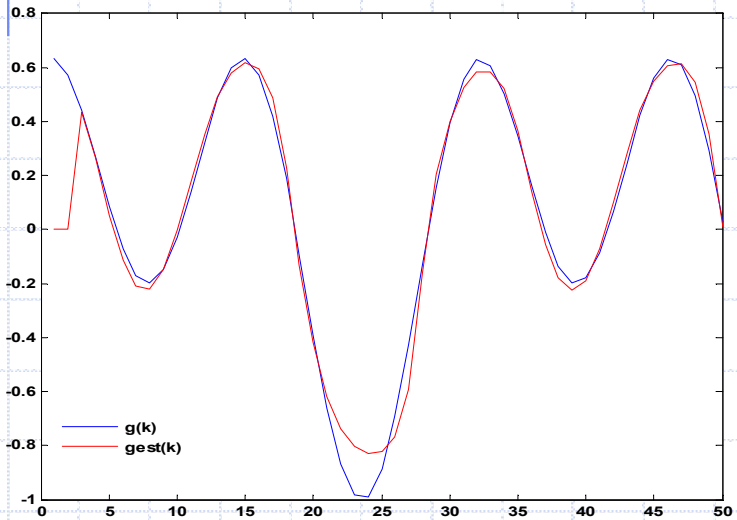
$$\Delta w_{21} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{32} f'(v_2(k)) g(k-1)$$

$$\Delta w_{22} = \mu(g(k) - \hat{g}(k)) f'(v(k)) w_{32} f'(v_2(k)) g(k-2)$$

Résultats

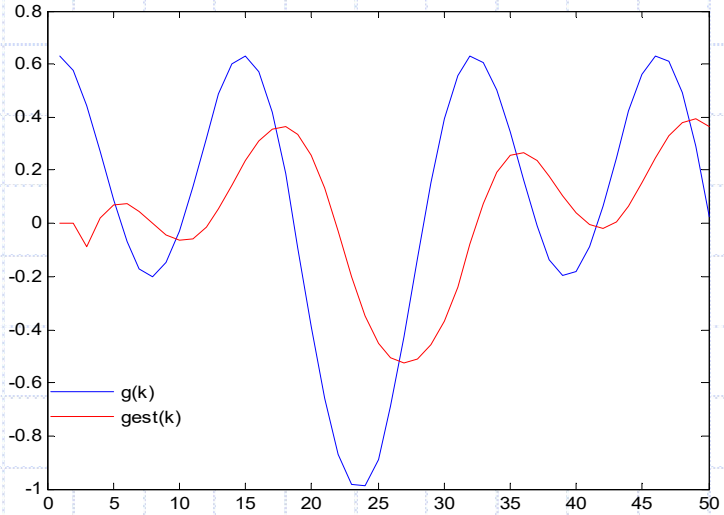
Evolution de g(k) (k=1:50)





Après 15 passages

Evolution de g(k) et la sortie du neurone



Après un seul passage

μ=0.25. Fonction d'activation:

$$f(v(k)) = \frac{1-e^{-2v(k)}}{1+e^{-2v(k)}}$$

Valeurs retrouvées après 15 passages:

$$w_{10} = 0.074,\ w_{11} = -1.375,\ w_{12} = 0.764$$

$$w_{20} = -0.007,\ w_{21} = 0.259,\ w_{22} = 0.061$$

$$w_{30} = 0.15,\ w_{31} = -1.57,\ w_{32} = 0.225$$

Modélisation des systèmes non linéaires par les réseaux de neurones artificiels

Système: $y(k) = h(y(k-1),\ldots,y(k-n),u(k-1),\ldots,u(k-m))$

h: $fonction\ \ non\ \ linéaire$

La sortie s'exprime d'une façon non linéaire en fonction des anciennes mesures.

$y(k)$ dépend de: $y(k\text{-}1),\ \ldots,\ y(k\text{-}n)$ et de $u(k\text{-}1),\ \ldots,\ u(k\text{-}m)$.

Système:

$$y(k) = h(y(k-1),\ldots,y(k-n),u(k-1),\ldots,u(k-m))$$

Exemples:

$$y(k) = y(k-1)u(k-2) + \left(y(k-2)\right)^3 + \sqrt{1+2u(k-1)-3y(k-2)}$$

$$y(k) = \frac{y(k-1)}{1+\left(y(k-2)\right)^3} + \left(u(k-2)\right)^2$$

Généralement, la fonction $h(.)$ est inconnue.

On possède un fichier de mesures obtenues du système réel.

Comment peut-on déterminer le modèle neuronal du système?

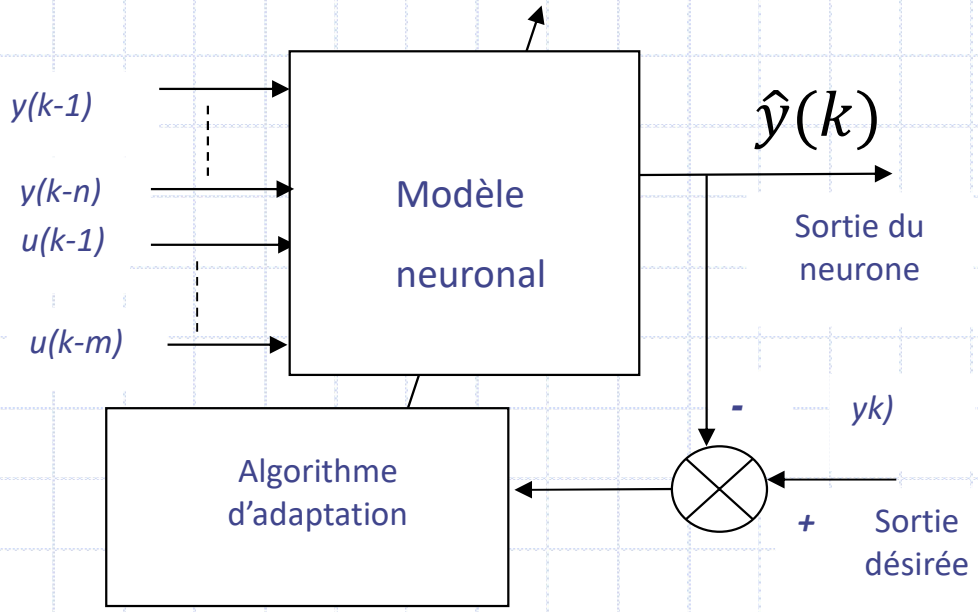
Modèle neuronal:

Mesures disponibles:

k	1	2	$\ldots$	Nm
$y(k)$	*	*	$\ldots$	*
$u(k)$	*	*	$\ldots$	*

Modèle basé sur les neurones

- Entrées: anciennes mesures de la sortie et de l'entrée
- Couche(s) cachée(s) (1: couche, 2: couches, ou plus)
- Couche de sortie



Modèle neuronal

Sortie du modèle:

$$\hat{y}(k) = NN(y(k-1),\ldots,y(k-n)\ \ u(k-1),\ldots,u(k-m))$$

NN: le réseau de neurone avec des pondérations inconnues,

Critère à minimiser:

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

$y(k)$: sortie du système réel

$\hat{y}(k)$: sortie du modèle neuronal

$$w_{ij}(k) = w_{ij}(k-1) + \Delta w_{ij}$$

$$\Delta w_{ij} = -\eta \frac{\partial J}{\partial w_{ij}} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_{ij}}$$

Exemple: système non linéaire

$$y(k) = -0.8\frac{(u(k-1))^2}{1+(u(k-1))^2}y(k-2) + 0.033y(k-1) + 0.5u(k-1)$$

Modèle neuronal

- ✓ Modèle avec un seul neurone
  - deux entrées $y(k-1)$ $u(k-1)$
  - fonction d'activation non linéaire

- ✓ Modèle avec une couche cachée
  - ✓ une couche cachée à deux neurones
  - ✓ une couche cachée avec dix neurones
  - ✓ une de sortie avec un seul neurone
  - ✓ deux entrées $y(k-1)$ $u(k-1)$
  - ✓ fonction d'activation non linéaire

Exemple: système non linéaire

$$y(k) = -0.8\frac{(u(k-1))^2}{1+(u(k-1))^2}y(k-2) + 0.033y(k-1) + 0.5u(k-1)$$

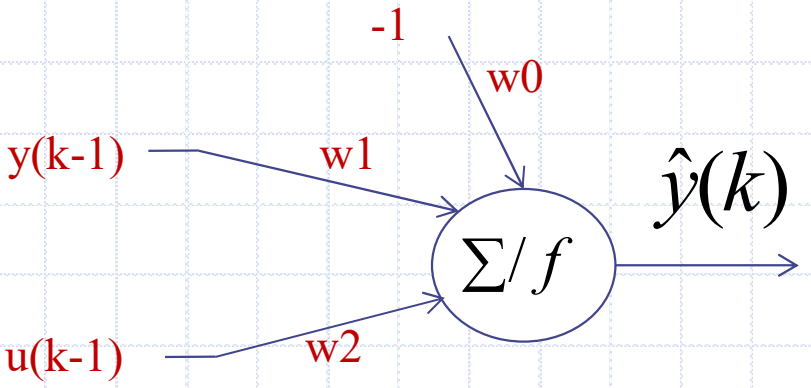
Modèle: utilisation d'un seul neurone

- deux entrées $y(k-1)$ $u(k-1)$
- fonction d'activation non linéaire

$$v = -w_0 + w_1 y(k-1) + w_2 u(k-1)$$

$$\hat{y}(k) = f_2(v)$$

$$f_2(v) = \frac{1-e^{-2v}}{1+e^{-2}}$$



modèle:

$$v = -w_0 + w_1 y(k-1) + w_2 u(k-1) \qquad \hat{y}(k) = f_2(v)$$

Critère à minimiser:

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids du neurone:

$$\Delta w_i = -\mu \frac{\partial J}{\partial w_i} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_i} \qquad i = 0, 1, 2$$

$$\frac{\partial \hat{y}(k)}{\partial w_i} = \frac{\partial \hat{y}(k)}{\partial v} \frac{\partial v}{\partial w_i} = f_2'(v) e_i(k)$$

$$e_0(k) = -1 \quad \forall k$$

$$e_1(k) = y(k-1)$$

$$e_2(k) = u(k-1)$$

$$\Delta w_i = \eta\left(y(k) - \hat{y}(k)\right) f_2'(v) e_i(k) \quad i = 0,1,2$$

$$w_i(k) = w_i(k-1) + \Delta w_i \qquad i = 0, 1, 2$$

Programme (Octave)

```
clear all
close all
% Package control
pkg load control
% Package optimisation
 pkg load optim

N=100;    % nombre de mesures
% Entrée du système
u= rand(1,N)-0.5;

% Calcul de la sortie du système non linéaire
y(1)=0; y(2)=0;
for k=3:N
    s=u(k-1)^2/(1+u(k-1)^2);
    y(k)=-0.8*s*y(k-2)+0.033*y(k-1)+0.5*u(k-1);
end
figure(1)
plot(u)
figure(2)
plot(y,'r')
mu=0.2;          % pas d'adaptation

% Initialisation des paramètres du modèle
w0=0.1; w1=-0.1; w2=0.1;

% nombre maximal de passages
pmax=30;

yest(1)=y(1);
```

```
% Apprentissage
for p=1:pmax
    for k=2:N
        % sortie du neurone
        v=-w0+w1*y(k-1)+w2*u(k-1);
        yest(k)=tanh(v);
        e(k)=y(k)-yest(k);

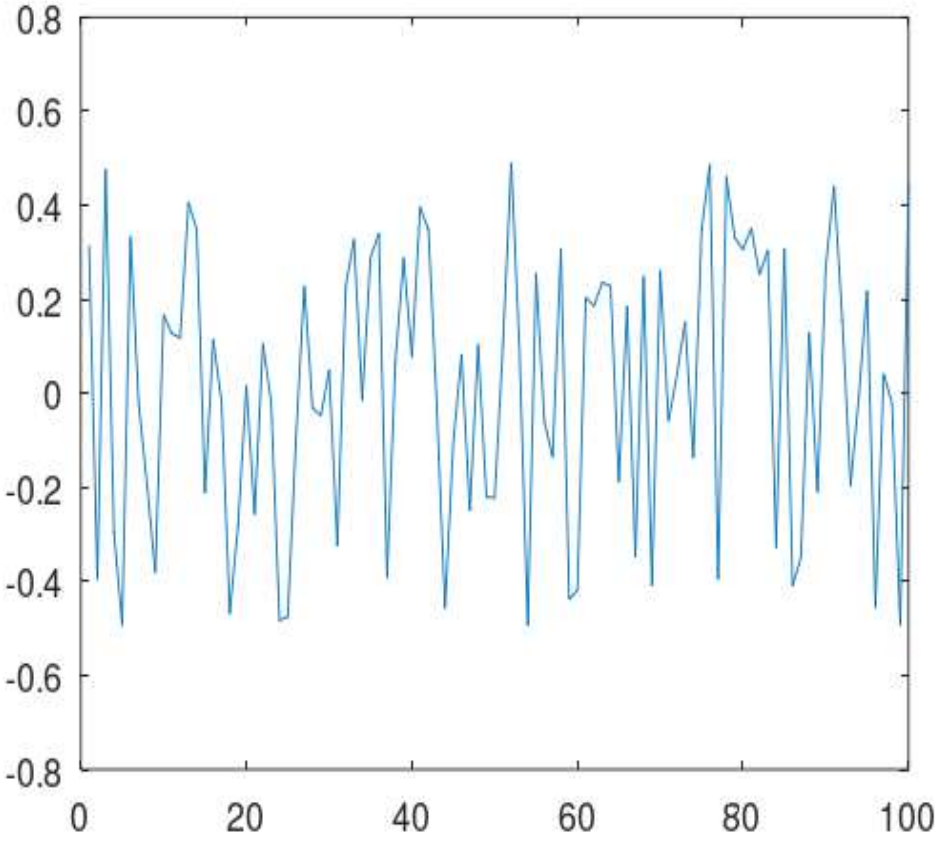
        % Termes de correction
        df = deriv(@tanh, v);     % calcul de tanh'(v)
        dw0=mu*(y(k)-yest(k))*df*(-1);
        dw1=mu*(y(k)-yest(k))*df*y(k-1);
        dw2=mu*(y(k)-yest(k))*df*u(k-1);

        % Adaptation des pondØrations
        w0=w0+dw0;
        w1=w1+dw1;
        w2=w2+dw2;
    end
   % critŁre : somme des erreurs au carrØ
   J(p)=e*e';
end
figure(3)
plot(J)    % critŁre de performance
grid

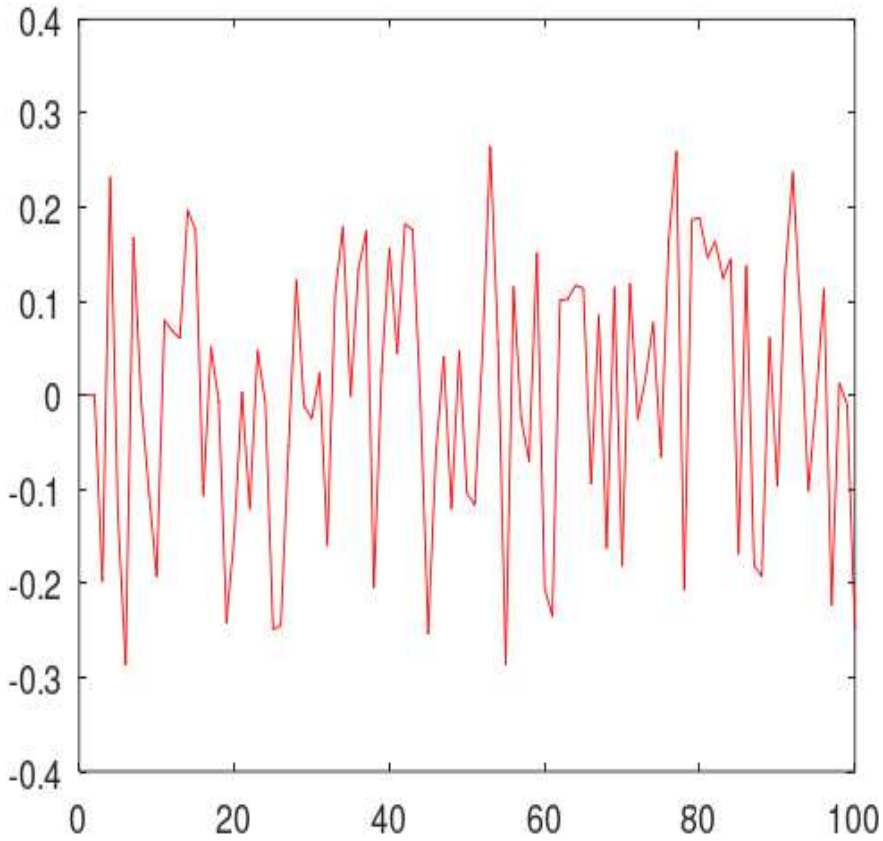
figure(4)
plot(y,'r');
hold on
plot(yest);
grid
```

Résultats

Evolution de l'entrée *u(k)* et de la sortie du système *y(k)*, L'entrée est une séquence aléatoire entre -1 et 1. Le nombre de mesures est N=100.



Entrée u



Sortie y

Résultats

- Paramètre μ=0,2.
- Fonction d'activation du neurone de sortie (fonction tangente hyperbolique):

$$f_2(v) = \frac{1 - e^{-2v}}{1 + e^{-2v}}$$

Valeurs retrouvées après 30 passages:

$$w_0 = -5\ 10^{-3} \quad w_1 = 0{,}032 \quad w_2 = 0{,}508$$

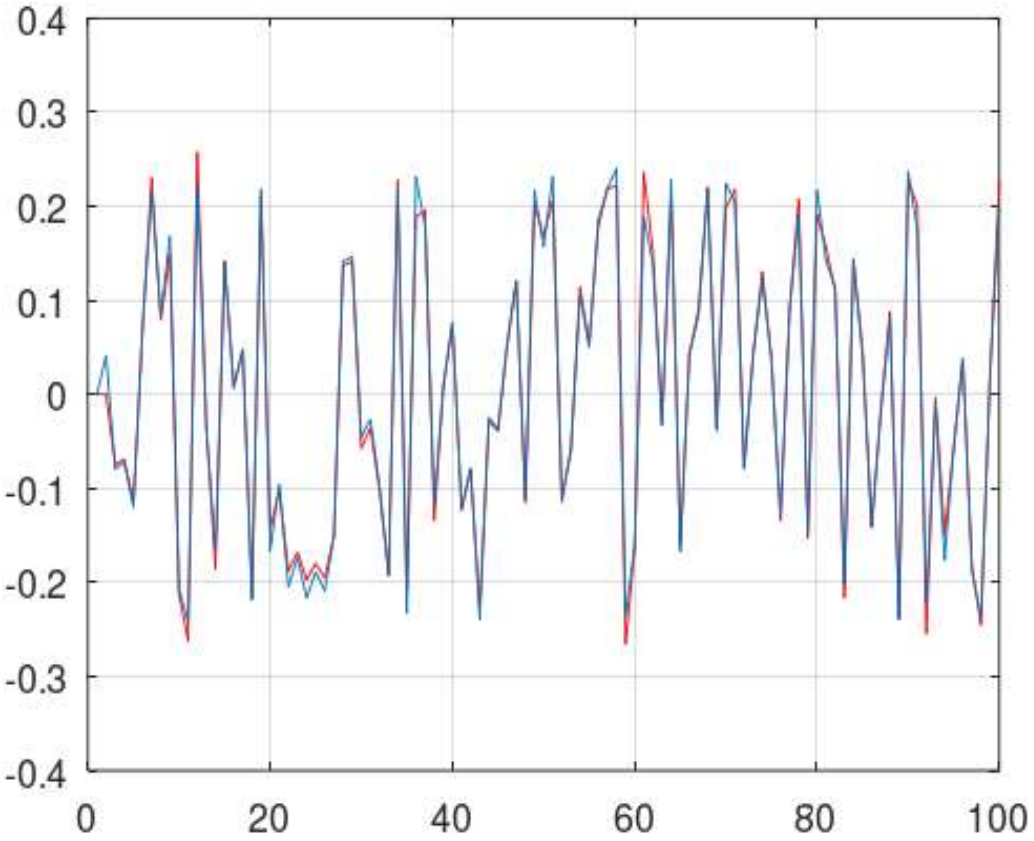
Modèle neuronal

$$\begin{aligned} v &= -w_0 + w_1 y(k-1) + w_2 u(k-1) \\ &= 5\ 10^{-3} + 0{,}032 y(k-1) + 0{,}508 u(k-1) \end{aligned}$$

$$\hat{y}(k) = f_2(v)$$

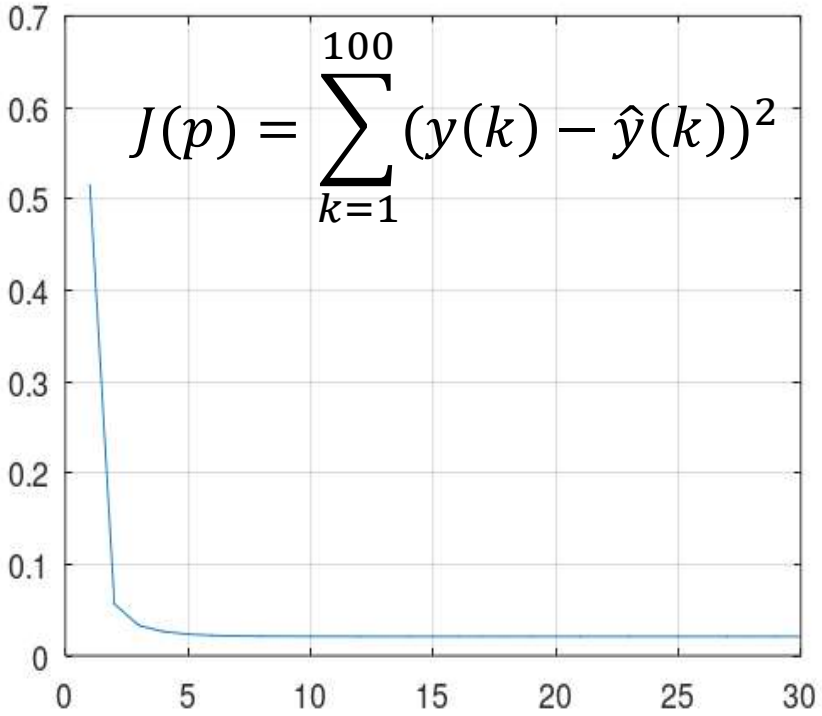
Résultats

Evolution de la sortie du système et de la sortie du modèle neuronal.



Evolution de la somme des erreurs au carré en fonction de passages

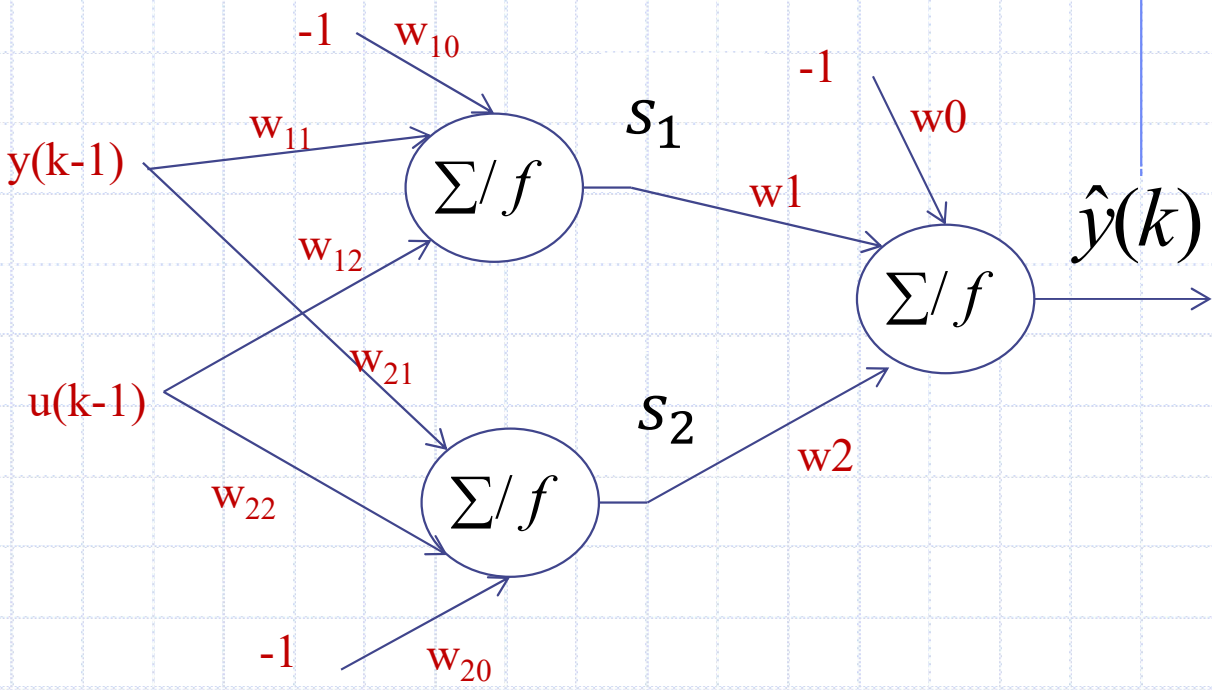
$$J(p) = \sum_{k=1}^{100} (y(k) - \hat{y}(k))^2$$



$$J(30) = 0{,}02$$

Modèle: un réseau de neurones formé par:

- deux entrées $y(k-1)$ $u(k-1)$
- une couche cachée à deux neurones
- une couche de sortie avec un seul neurone.



$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \quad j = 1,2$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + w_2 s_2(k) \qquad \hat{y}(k) = f_2(v(k))$$

Modèle:

$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \qquad j = 1,2$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + w_2 s_2(k)$$

$$\hat{y}(k) = f_2(v(k))$$

Critère

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids du neurone de sortie:

$$\Delta w_i = -\mu \frac{\partial J}{\partial w_i} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_i} \qquad i = 0,1,2$$

$$\frac{\partial \hat{y}(k)}{\partial w_i} = \frac{\partial \hat{y}(k)}{\partial v} \frac{\partial v}{\partial w_i} = f_2'(v) e_i(k)$$

$$e_0(k) = -1 \quad \forall k$$

$$e_1(k) = y(k-1)$$

$$e_2(k) = u(k-1)$$

$$\Delta w_i = \eta\left(y(k) - \hat{y}(k)\right) f_2'(v) e_i(k) \qquad i = 0,1,2$$

$$w_i(k) = w_i(k-1) + \Delta w_i \qquad i = 0,1,2$$

$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \quad j = 1,2 \qquad s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + w_2 s_2(k) \qquad \hat{y}(k) = f_2(v(k))$$

Critère

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids des neurones de la couche cachée:

$$\Delta w_{ji} = -\mu \frac{\partial J}{\partial w_{ji}} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_{ji}} \qquad j = 1,2 \quad i = 0,1,2$$

$$\frac{\partial \hat{y}(k)}{\partial w_{ji}} = \frac{\partial \hat{y}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial s_j(k)} \frac{\partial s_j(k)}{\partial v_j(k)} \frac{\partial v_j(k)}{\partial w_{ji}}$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j0}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))(-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j2}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, u(k-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j1}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, y(k-1)$$

Résultats

- Paramètre μ=0,2.
- Fonction d'activation du neurone de sortie (fonction tangente hyperbolique):

$$f_2(v) = \frac{1 - e^{-2v}}{1 + e^{-2v}}$$

Valeurs retrouvées après 30 passages:

Modèle neuronal

$$w0 = -0.023592 \quad w1 = -0.6712 \quad w2 = 0.2355$$

$$w10 = 0.021642 \quad w11 = -0.059432 \quad w12 = -0.6872$$

$$w20 = 0.1653 \quad w21 = -0.041310 \quad w22 = 0.2488$$

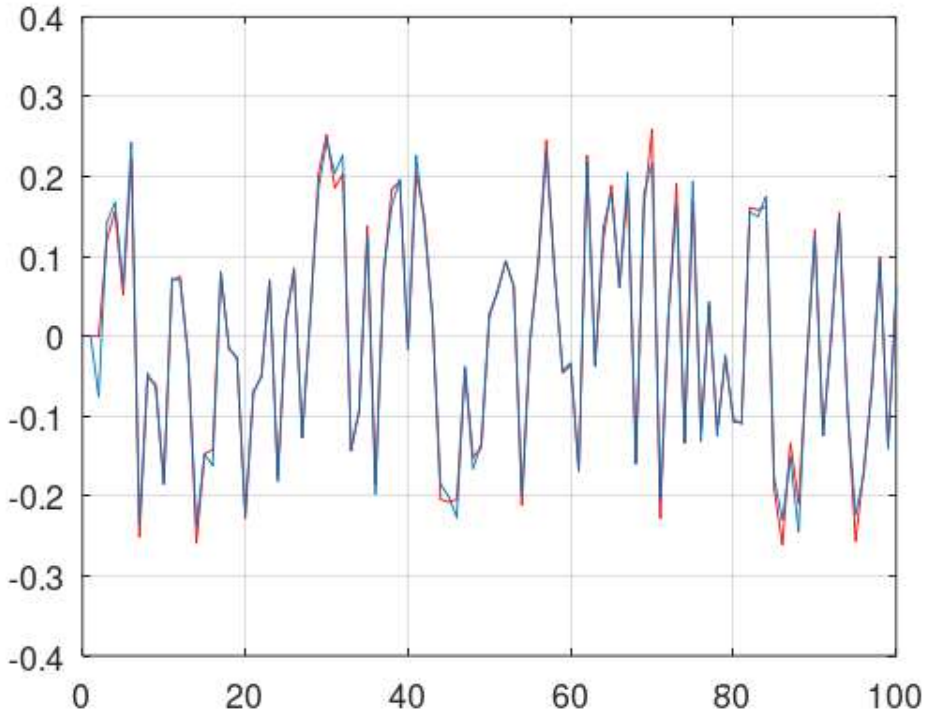
$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \qquad j = 1,2$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + w_2 s_2(k) \qquad \hat{y}(k) = f_2(v(k))$$

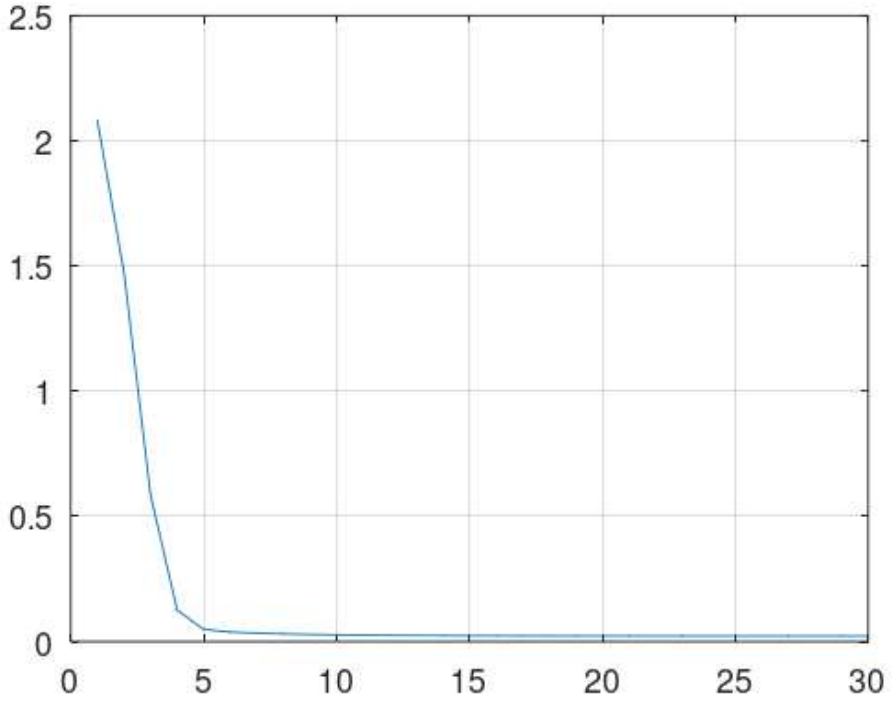
Résultats

Evolution de la sortie du système et de la sortie du modèle neuronal.



Evolution de la somme des erreurs au carré en fonction de passages

$$J(p) = \sum_{k=1}^{100} (y(k) - \hat{y}(k))^2$$



$$J(30) = 0{,}05$$

Exemple: système non linéaire

$$y(k) = -0.8\frac{(u(k-1))^2}{1+(u(k-1))^2}y(k-2) + 0.033y(k-1) + 0.5u(k-1)$$

Modèle: un réseau de neurones formé par:

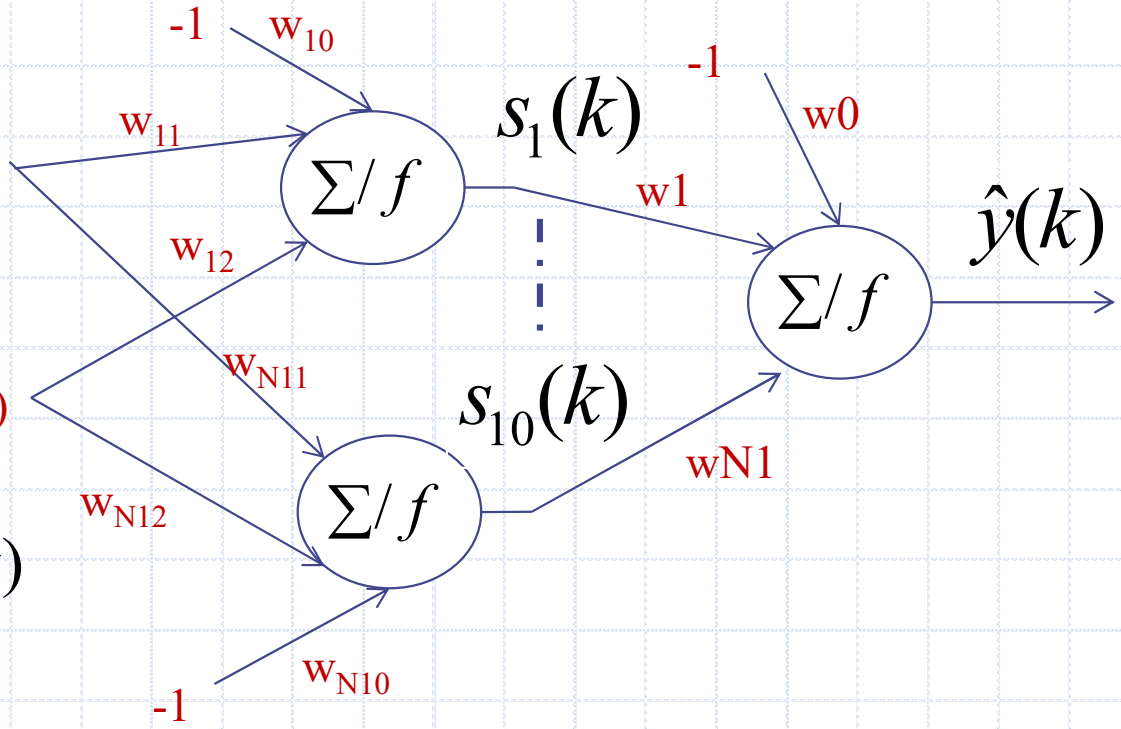
- deux entrées $y(k-1)$ $u(k-1)$
- une couche cachée à 10 neurones $N_1 = 10$
- une couche de sortie avec un seul neurone.

$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \quad j = 1,\ldots,N_1$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + \ldots + w_{N_1} s_{N_1}(k)$$

$$\hat{y}(k) = f_2(v(k))$$



Exemple: $v_j(k) = -w_{j0} + w_{j1} y(k-1) + w_{j2} u(k-1) \quad j = 1,\ldots,N_1$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + \ldots + w_{N_1} s_{N_1}(k)$$

$$\hat{y}(k) = f_2(v(k))$$

Critère

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids du neurone de sortie:

$$\Delta w_i = -\mu \frac{\partial J}{\partial w_i} = \eta (y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_i} \qquad i = 0,\ldots,N_1$$

$$\frac{\partial \hat{y}(k)}{\partial w_i} = \frac{\partial \hat{y}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial w_i} = f_2^{'}(v(k) s_i(k) \qquad s_0(k) = -1 \ \ \forall k$$

$$\Delta w_i = \eta (y(k) - \hat{y}(k)) f_2^{'}(v(k)) s_i(k) \quad i = 0,\ldots,N_1$$

$$w_i(k) = w_i(k-1) + \Delta w_i \qquad i = 0,1,\ldots,N_1$$

$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \quad j = 1,\ldots,N_1$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + \ldots + w_{N_1} s_{N_1}(k)$$

$$\hat{y}(k) = f_2(v(k))$$

Critère

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids des neurones de la couche cachée:

$$\Delta w_{ji} = -\mu \frac{\partial J}{\partial w_{ji}} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_{ji}} \qquad j = 1,\ldots,N_1 \quad i = 0,1,2$$

$$\frac{\partial \hat{y}(k)}{\partial w_{ji}} = \frac{\partial \hat{y}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial s_j(k)} \frac{\partial s_j(k)}{\partial v_j(k)} \frac{\partial v_j(k)}{\partial w_{ji}}$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j0}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))(-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j2}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, u(k-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j1}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, y(k-1)$$

$$v_j(k) = -w_{j0} + w_{j1}y(k-1) + w_{j2}u(k-1) \quad j = 1,\ldots,N_1$$

$$s_j(k) = f_1(v_j(k))$$

$$v(k) = -w_0 + w_1 s_1(k) + \ldots + w_{N_1} s_{N_1}(k)$$

$$\hat{y}(k) = f_2(v(k))$$

Critère

$$J = \frac{1}{2}\left(y(k) - \hat{y}(k)\right)^2$$

Adaptation de poids des neurones de la couche cachée:

$$\Delta w_{ji} = -\mu \frac{\partial J}{\partial w_{ji}} = \eta(y(k) - \hat{y}(k)) \frac{\partial \hat{y}(k)}{\partial w_{ji}} \qquad j = 1,\ldots,N_1 \quad i = 0,1,2$$

$$\frac{\partial \hat{y}(k)}{\partial w_{ji}} = \frac{\partial \hat{y}(k)}{\partial v(k)} \frac{\partial v(k)}{\partial s_j(k)} \frac{\partial s_j(k)}{\partial v_j(k)} \frac{\partial v_j(k)}{\partial w_{ji}}$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j0}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))(-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j2}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, u(k-1)$$

$$\frac{\partial \hat{y}(k)}{\partial w_{j1}} = f_2'(v(k))\, w_j\, f_1'(v_j(k))\, y(k-1)$$

Adaptation de poids des neurones de la couche cachée:

$$\Delta w_{j0} = \eta(y(k) - \hat{y}(k)) f_2^{'}(v(k)) w_j f_1^{'}(v_j(k))(-1)$$

$$j = 1,\ldots, N_1$$

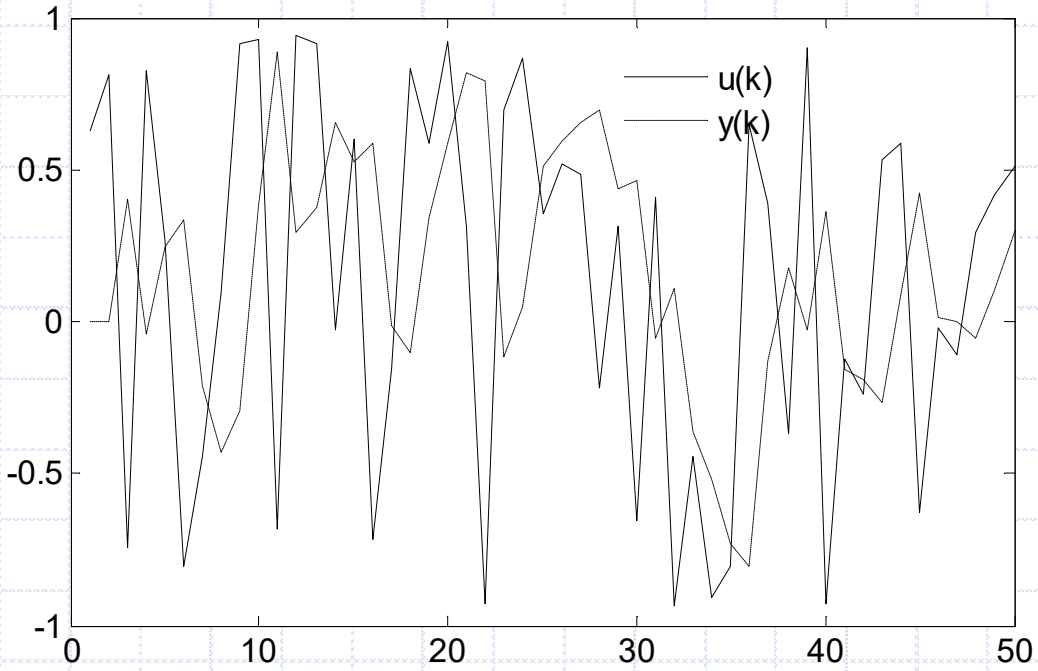
$$\Delta w_{j1} = \eta(y(k) - \hat{y}(k)) f_2^{'}(v(k)) w_j f_1^{'}(v_j(k)) y(k-1)$$

$$\Delta w_{j2} = \eta(y(k) - \hat{y}(k)) f_2^{'}(v(k)) w_j f_1^{'}(v_j(k)) u(k-1)$$

$$w_{ji} = w_{ji} + \Delta w_{ji}$$

Résultats

Evolution de l'entrée *u(k)* et de la sortie du système *y(k)*, L'entrée est une séquence aléatoire entre -1 et 1. Le nombre de mesures est N=50.



Résultats

Apprentissage du modèle neuronal.

μ=0,1.

- Fonction d'activation des neurones de la couche cachée (fonction non linéaire):

$$f_1(v(k)) = \frac{1 - e^{-2v(k)}}{1 + e^{-2v(k)}}$$

- Fonction d'activation du neurone de sortie (fonction linéaire):

$$f_2(v(k)) = v(k)$$

Valeurs retrouvées après 10 passages:

Neurones cachés

w_{j0} = [5.0862 -2.8734 -1.6731 1.2479 2.0088 -0.5774 1.8545 -4.1515 5.1003 -4.4444]

W_1 = [-2.3070 2.6158; 3.7230 -3.2367;4.2229 2.6462; -2.5439 1.5002;-4.7879 3.2810;0.2275 3.9862;5.1520 3.6620;0.1137 -3.3557;-2.2720 -3.3015;-3.3476 -2.8881]

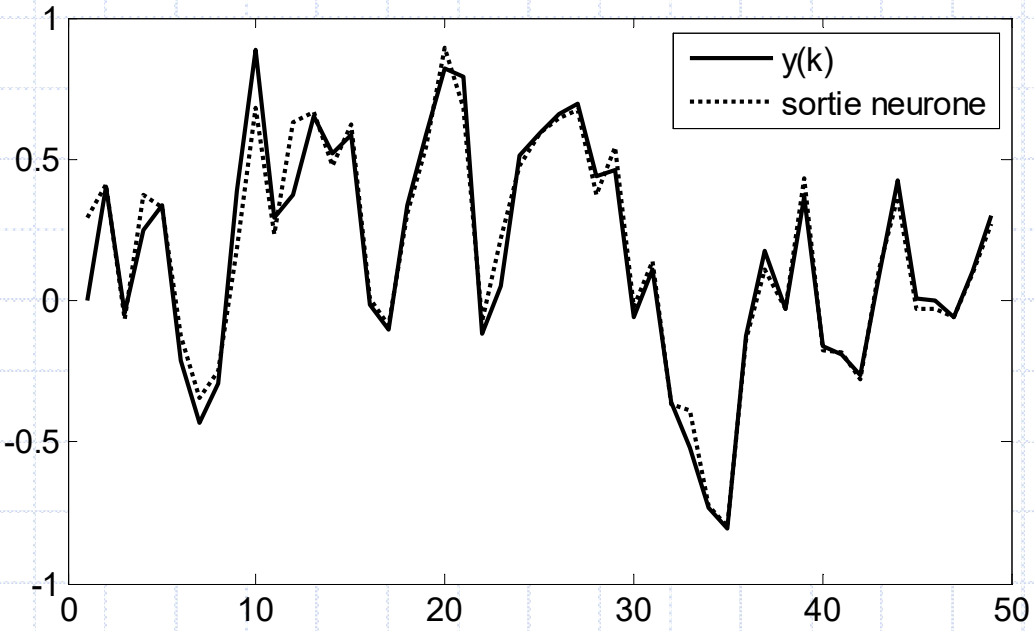
Neurone de sortie

W = [0.3150 -0.1296 0.2486 -0.2344 0.0522 0.1165 0.1690 -0.0399 -0.6324 -0.2772]

w_0 = 0.0926

Résultats

Evolution de la sortie du système et de la sortie du modèle neuronal.



On constate que la sortie du modèle neuronal suit convenablement la sortie du système.

FIN